Universidade

Departamento de Ciência da Computação

Programação Concorrente

Trabalho 1 — Versão 2.0

Logger Assíncrono com Semáforos POSIX

Produtor-Consumidor com Buffer Limitado e Logger Assíncrono

Aluno: Gustavo Choueiri

Data: Novembro de 2026

Relatório apresentado como requisito parcial da disciplina de Programação Concorrente.

Sumário

1 Introdução **1**

2 Formalização do Problema Proposto **1**

2.1 Definição 1

2.2 Condições de Corrida Identificadas 2

2.3 Invariantes que a Solução deve Garantir 2

3 Descrição do Algoritmo Desenvolvido **3**

3.1 Arquitetura do Sistema Refatorado 3

3.2 Visão Geral da Arquitetura 3

3.3 Estruturas de Dados 4

3.4 Mecanismos de Sincronização 4

3.5 Protocolo da Thread Produtora 4

3.6 Logger Assíncrono com Semáforos POSIX 5

3.7 Protocolo da Thread Consumidora 6

3.8 Correção da Solução 6

3.9 Resultado da Execução 7

4 Conclusão **8**

A Código-Fonte dos Módulos Principais **11**

1 Introdução

A programação concorrente é um paradigma fundamental na computação moderna, presente desde sistemas embarcados de baixo nível até servidores de alta escala na nuvem. A capacidade de executar múltiplas tarefas simultaneamente aumenta significativamente o desempenho de aplicações; contudo, introduz uma classe de desafios delicada: a **sincronização entre processos e threads** que compartilham recursos na memória.

Quando múltiplas threads acessam e modificam uma estrutura de dados compartilhada sem nenhum mecanismo de controle, surgem as chamadas **condições de corrida** (*race conditions*). Uma condição de corrida ocorre quando o resultado correto de um programa depende da ordem de execução relativa entre as threads, tornando o comportamento da aplicação não determinístico, difícil de depurar e potencialmente catastrófico em sistemas críticos [1].

Para tratar esse problema, a biblioteca padrão POSIX define a interface `Pthreads` (*POSIX Threads*), que fornece primitivas robustas de sincronização: **mutexes** (*mutual exclusion locks*), **variáveis de condição** e **semáforos POSIX** (`sem_t`). Essas primitivas permitem que o programador delimite regiões críticas do código, garantindo exclusão mútua e comunicação segura entre threads [3].

O presente trabalho, em sua **Versão 2.0**, expande a solução original do Problema Produtor-Consumidor com duas contribuições principais: (1) a **refatoração do código em múltiplos módulos** (`.h`/`.c`), com responsabilidades claramente separadas; e (2) a adição de uma **camada de logger assíncrono** sincronizada exclusivamente com **semáforos POSIX** (`sem_t`), demonstrando essa primitiva em paralelo com o mutex e as variáveis de condição já utilizados no buffer principal. O relatório está organizado da seguinte forma: a Seção 2 formaliza o problema proposto; a Seção 3 descreve o algoritmo e a nova arquitetura; e a Seção 4 apresenta as considerações finais.

2 Formalização do Problema Proposto

2.1 Definição

O **Problema Produtor-Consumidor com Buffer Limitado** (também denominado *Bounded-Buffer Problem*) é um dos problemas de sincronização mais estudados na

literatura [4]. Ele modela situações reais em que componentes de um sistema produzem dados em taxas distintas de seu consumo, como *pipelines* de processamento, filas de mensagens e *spoolers* de impressão.

Formalmente, o problema é definido por um conjunto de entidades e restrições:

- **Buffer compartilhado:** uma estrutura circular de capacidade N posições, acessível por todas as threads.
- **Threads Produtoras** $(P_1, P_2, \ldots, P_m)$: cada uma gera itens e os deposita no buffer.
- **Threads Consumidoras** $(C_1, C_2, \ldots, C_k)$: cada uma retira itens do buffer e os processa.
- **Restrição de capacidade:** o número de itens no buffer a qualquer instante t satisfaz $0 \leq |B(t)| \leq N$.

2.2 Condições de Corrida Identificadas

Sem mecanismos de sincronização, as seguintes condições de corrida emergem:

1. **Inserção concorrente:** dois produtores calculam simultaneamente o índice de cauda (`cauda`) e sobrescrevem o mesmo slot do buffer, perdendo um dos itens produzidos.
2. **Remoção concorrente:** dois consumidores leem simultaneamente o mesmo item da cabeça (`cabeca`), processando o mesmo valor duas vezes e pulando o próximo.
3. **Overflow:** um produtor insere um item quando o buffer está cheio ($|B(t)| = N$), corrompendo um item ainda não consumido.
4. **Underflow:** um consumidor retira um item quando o buffer está vazio ($|B(t)| = 0$), lendo um valor inválido da memória.

2.3 Invariantes que a Solução deve Garantir

A solução correta deve assegurar as seguintes propriedades:

Propriedade	Descrição
Exclusão Mútua	No máximo uma thread acessa o buffer por vez.
Ausência de Deadlock	O sistema sempre progride; nenhuma thread espera indefinidamente por recurso que nunca será liberado.
Ausência de Starvation	Todo produtor eventualmente insere seu item; todo consumidor eventualmente retira um item.
Consistência do Buffer	$0 \leq \lvert B(t) \rvert \leq N$ em qualquer instante.

3 Descrição do Algoritmo Desenvolvido

3.1 Arquitetura do Sistema Refatorado

A versão 2.0 adota uma arquitetura modular com responsabilidades bem separadas. O projeto é composto pelos seguintes arquivos:

Arquivo	Responsabilidade
`config.h`	Constantes globais (`TAMANHO_BUFFER`, `NUM_PRODUTORES`, etc.)
`buffer.h/.c`	Buffer circular + mutex + variáveis de condição
`logger.h/.c`	Fila de log + semáforos POSIX + thread de log
`produtor.h/.c`	Lógica da thread produtora
`consumidor.h/.c`	Lógica da thread consumidora
`main.c`	Inicialização, criação de threads e encerramento
`Makefile`	Compilação de todos os módulos

3.2 Visão Geral da Arquitetura

A solução é composta por quatro camadas lógicas:

- **Threads Produtoras** (P_1, P_2, P_3): geram itens e os inserem no buffer compartilhado; delegam o log via `log_evento()`.
- **Buffer Circular Compartilhado** (5 posições): estrutura de dados central protegida por mutex e variáveis de condição.
- **Threads Consumidoras** (C_1, C_2, C_3): retiram e processam os itens do buffer; delegam o log via `log_evento()`.
- **Thread de Logger Assíncrono**: consome eventos da fila de log e os exibe com timestamp; sincronizada por semáforos POSIX.

3.3 Estruturas de Dados

O buffer compartilhado é implementado como um **vetor circular** de inteiros de tamanho fixo $N = 5$. Dois ponteiros — `cabeca` e `cauda` — percorrem o vetor de forma circular usando aritmética modular, evitando realocações dinâmicas. A variável `quantidade` mantém o número de itens presentes no buffer em qualquer instante:

```
cabeca = (cabeca + 1) % TAMANHO_BUFFER
cauda = (cauda + 1) % TAMANHO_BUFFER
```

A fila de log é também um vetor circular separado de `EventoLog` com capacidade `TAMANHO_FILA_LOG = 20`.

3.4 Mecanismos de Sincronização

A tabela a seguir resume os três mecanismos utilizados na versão 2.0:

Primitiva	**Onde é usada**	**Finalidade**
`pthread_mutex_t`	Buffer principal	Exclusão mútua na região crítica de inserção/remoção
`pthread_cond_t`	Buffer principal	Bloquear produtores (buffer cheio) e consumidores (buffer vazio)
`sem_t` (POSIX)	Fila de log	Controlar espaços livres e itens pendentes na fila de eventos

3.5 Protocolo da Thread Produtora

Cada thread produtora segue o protocolo abaixo:

1. Simular tempo de produção (`usleep`).
2. Invocar `buffer_inserir(id)`, que internamente:
   (a) Adquire o mutex (`pthread_mutex_lock`).
   (b) **Enquanto** o buffer estiver cheio, envia `LOG_BUF_CHEIO` ao logger e aguarda na variável de condição `nao_cheio`.
   (c) Insere o item no slot `cauda` e avança o ponteiro.
   (d) Incrementa `quantidade`.

(e) Sinaliza a variável `nao_vazio`.

(f) Libera o mutex.

(g) Envia evento `LOG_PRODUZIU` ao logger assíncrono.

O uso do laço `while` ao invés de `if` na etapa 2b é essencial para lidar com *spurious wakeups* — situação prevista pelo padrão POSIX em que a thread pode ser desbloqueada sem que a condição esperada tenha se tornado verdadeira [3].

3.6 Logger Assíncrono com Semáforos POSIX

O módulo de log (`logger.c`) implementa uma fila produtora-consumidora própria, onde *produtores são as threads do sistema principal* e o *único consumidor é a thread de log*. A sincronização usa **exclusivamente semáforos POSIX** (`sem_t`):

- `sem_t sem_espacos_log`: semáforo contador inicializado com `TAMANHO_FILA_LOG` (20). Representa espaços livres na fila.
- `sem_t sem_itens_log`: semáforo contador inicializado com 0. Representa itens pendentes para exibição.
- `pthread_mutex_t mutex_log`: protege o índice circular da fila de log durante leitura e escrita.

O fluxo de envio de um evento via `log_evento()` é:

1. Chama `sem_trywait(&sem_espacos_log)` — não bloqueia.
2. Se retornar `EAGAIN` (fila cheia), descarta o evento silenciosamente.
3. Caso contrário, preenche o evento com timestamp e escreve na fila com proteção do `mutex_log`.
4. Chama `sem_post(&sem_itens_log)` para sinalizar a thread de log.

A thread de log executa em loop:

1. `sem_wait(&sem_itens_log)` — bloqueia enquanto fila vazia.
2. Adquire `mutex_log`, lê o evento, libera o mutex.

3. `sem_post(&sem_espacos_log)` — libera uma posição.

4. Exibe o evento formatado com timestamp em milissegundos.

5. Para ao receber evento sentinela `LOG_SHUTDOWN`.

O uso de `sem_trywait` (não-bloqueante) no envio garante que produtores e consumidores **nunca bloqueiam** aguardando o logger, preservando o desempenho do sistema principal.

3.7 Protocolo da Thread Consumidora

A thread consumidora segue o protocolo simétrico ao produtor:

1. Simular tempo de consumo (`usleep`).

2. Invocar `buffer_retirar(id)`, que internamente:

   (a) Adquire o mutex.

   (b) **Enquanto** o buffer estiver vazio, envia `LOG_BUF_VAZIO` e aguarda na variável `nao_vazio`.

   (c) Lê o item no slot `cabeca` e avança o ponteiro.

   (d) Decrementa `quantidade`.

   (e) Sinaliza `nao_cheio`.

   (f) Libera o mutex.

   (g) Envia evento `LOG_CONSUMIU` ao logger assíncrono.

3.8 Correção da Solução

A tabela a seguir mapeia cada propriedade de correção exigida ao mecanismo que a garante na implementação:

Propriedade	Mecanismo
Exclusão Mútua	`pthread_mutex_lock/unlock` envolve toda a região crítica.
Ausência de Overflow	Laço `while (quantidade == N)` bloqueia o produtor antes de inserir.
Ausência de Underflow	Laço `while (quantidade == 0)` bloqueia o consumidor antes de retirar.
Ausência de Deadlock	O mutex é liberado por `pthread_cond_wait` e os sinais são emitidos após cada inserção/remoção.
Ausência de Starvation	O escalonador POSIX garante progresso; `pthread_cond_signal` acorda pelo menos uma thread em espera.
Log não-bloqueante	`sem_trywait` descarta eventos se a fila de log estiver cheia, sem atrasar produtores nem consumidores.

3.9 Resultado da Execução

A execução do programa com 3 produtores, 3 consumidores e buffer de 5 posições produziu a saída abaixo, confirmando a completa sincronização e ausência de condições de corrida. O timestamp em milissegundos evidencia o caráter assíncrono do logger:

```
=== Produtor-Consumidor v2.0 com Logger Assincrono ===
Buffer:  5 posicoes | 3 produtores | 3 consumidores
[ 158.6ms] [Produtor 3] Inseriu:  1 | Buffer:  1/5
[ 212.4ms] [Produtor 2] Inseriu:  2 | Buffer:  2/5
[ 241.5ms] [Produtor 1] Inseriu:  3 | Buffer:  3/5
[ 281.4ms] [Consumidor 3] Retirou:  1 | Buffer:  2/5
[ 366.4ms] [Consumidor 1] Retirou:  2 | Buffer:  1/5
[ 376.8ms] [Produtor 1] Inseriu:  4 | Buffer:  2/5
[ 379.8ms] [Produtor 3] Inseriu:  5 | Buffer:  3/5
[ 540.5ms] [Consumidor 2] Retirou:  3 | Buffer:  2/5
[ 552.5ms] [Produtor 2] Inseriu:  6 | Buffer:  3/5
[ 596.9ms] [Produtor 3] Inseriu:  7 | Buffer:  4/5
[ 620.0ms] [Produtor 1] Inseriu:  8 | Buffer:  5/5
[ 624.6ms] [Consumidor 3] Retirou:  4 | Buffer:  4/5
[ 748.7ms] [Produtor 2] Inseriu:  9 | Buffer:  5/5
[ 759.1ms] [Produtor 3] Buffer cheio.  Aguardando...
```

```
[ 840.7ms] [Consumidor 1] Retirou:  5 | Buffer:  4/5
[ 840.8ms] [Produtor 3] Inseriu:  10 | Buffer:  5/5
[ 844.7ms] [Consumidor 2] Retirou:  6 | Buffer:  4/5
[ 1012.3ms] [Produtor 1] Inseriu:  11 | Buffer:  5/5
[ 1046.8ms] [Produtor 2] Buffer cheio.  Aguardando...
[ 1072.7ms] [Consumidor 3] Retirou:  7 | Buffer:  4/5
[ 1072.8ms] [Produtor 2] Inseriu:  12 | Buffer:  5/5
[ 1099.9ms] [Consumidor 1] Retirou:  8 | Buffer:  4/5
[ 1312.1ms] [Consumidor 2] Retirou:  9 | Buffer:  3/5
[ 1509.1ms] [Consumidor 1] Retirou:  10 | Buffer:  2/5
[ 1625.0ms] [Consumidor 3] Retirou:  11 | Buffer:  1/5
[ 1745.4ms] [Consumidor 2] Retirou:  12 | Buffer:  0/5
=== Execucao concluida com sucesso!  ===
```

A análise da saída demonstra precisamente as seguintes propriedades:

1. **Bloqueio em buffer cheio:** em 759,1 ms e 1046,8 ms, os produtores encontraram o buffer cheio (5/5) e suspenderam sua execução corretamente via variável de condição `nao_cheio`.

2. **Retomada pontual:** logo que um consumidor liberou vaga (em 840,7 ms e 1072,7 ms), o produtor suspenso acordou prontamente e concluiu a inserção.

3. **Consumo integral e esvaziamento:** exatamente 12 itens foram produzidos (3 produtores $\times$ 4 itens) e todos os 12 itens foram consumidos (3 consumidores $\times$ 4 itens), resultando na ocupação final de 0/5 no instante 1745,4 ms.

4. **Sequência correta de encerramento:** a thread principal aguardou o término de todos os produtores e em seguida de todos os consumidores antes de invocar `logger_shutdown()`, garantindo que nenhum registro de log fosse perdido ou truncado.

4 Conclusão

Este trabalho, em sua Versão 2.0, expandiu a solução original do Problema Produtor-Consumidor em duas dimensões fundamentais: a **refatoração modular** do código e a

adição de semáforos POSIX como nova primitiva de sincronização.

A refatoração separou claramente as responsabilidades em módulos independentes (`buffer`, `logger`, `produtor`, `consumidor`, `main`), facilitando a manutenção, o teste e a compreensão do sistema. O Makefile automatiza a compilação de todos os módulos.

O logger assíncrono demonstrou a aplicação prática de **semáforos POSIX** (`sem_t`) como alternativa às variáveis de condição: enquanto o buffer principal emprega o par mutex e condição, a fila de log combina os semáforos `sem_espacos_log` e `sem_itens_log` com o lock `mutex_log`. O uso de `sem_trywait` garantiu que as threads do sistema principal nunca bloqueiem aguardando o logger.

Como lições aprendidas, destacam-se:

- O uso do laço `while` ao invés de `if` ao aguardar uma variável de condição é indispensável para robustez frente a *spurious wakeups*.
- O padrão *lock* → *verificar* → *operar* → *sinalizar* → *unlock* deve ser aplicado de forma consistente para evitar janelas de vulnerabilidade.
- Semáforos POSIX são especialmente adequados para padrões produtor-consumidor simples, pois encapsulam diretamente o contador de itens e espaços sem a necessidade de um mutex separado para a condição.
- A combinação de múltiplas primitivas (mutex, variáveis de condição e semáforos) em um mesmo sistema requer atenção à ordem de encerramento para evitar deadlocks.

Como trabalhos futuros, sugere-se explorar o problema dos *Leitores-Escritores* e o *Jantar dos Filósofos* com as mesmas primitivas, bem como a comparação de desempenho entre mutex+condição e semáforos POSIX em cenários de alta contention.

Referências

[1] TANENBAUM, A. S.; BOS, H. **Modern Operating Systems**. 4. ed. Upper Saddle River: Pearson, 2015.

[2] SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10. ed. Hoboken: Wiley, 2018.

[3] KERRISK, M. **The Linux Programming Interface**. San Francisco: No Starch Press, 2010.

[4] DIJKSTRA, E. W. Cooperating Sequential Processes. In: *Programming Languages*, F. Genuys (ed.), Academic Press, New York, 1968, pp. 43–112.

[5] BUTTAZZO, G. C. **Hard Real-Time Computing Systems**. 3. ed. New York: Springer, 2011.

[6] THE OPEN GROUP. **POSIX.1-2017 – pthread.h**. Disponível em: https://pubs.opengroup.org/onlinepubs/9699919799/. Acesso em: nov. 2026.

A Código-Fonte dos Módulos Principais

config.h

```
#ifndef CONFIG_H
#define CONFIG_H

#define TAMANHO_BUFFER   5
#define NUM_PRODUTORES   3
#define NUM_CONSUMIDORES 3
#define ITENS_POR_THREAD 4
#define TAMANHO_FILA_LOG 20

#endif
```

buffer.h

```
#ifndef BUFFER_H
#define BUFFER_H

#include <pthread.h>
#include "config.h"

typedef struct {
  int dados[TAMANHO_BUFFER];
  int cabeca, cauda, quantidade;
  int proximo_valor;
} Buffer;

extern Buffer buffer_compartilhado;
extern pthread_mutex_t mutex_buffer;
extern pthread_cond_t  nao_cheio;
extern pthread_cond_t  nao_vazio;

void buffer_init(void);
void buffer_destroy(void);
int  buffer_inserir(int id_thread);
int  buffer_retirar(int id_thread);

#endif
```

logger.h

```
#ifndef LOGGER_H
#define LOGGER_H

typedef enum {
  LOG_PRODUZIU, LOG_CONSUMIU,
  LOG_BUF_CHEIO, LOG_BUF_VAZIO,
  LOG_SHUTDOWN
} TipoEvento;

typedef struct {
  TipoEvento tipo;
  int  thread_id;
  int  valor;
  int  quantidade_no_buffer;
  double timestamp_ms;
} EventoLog;

void logger_init(void);
void logger_iniciar_thread(void);
void log_evento(TipoEvento, int, int, int);
void logger_shutdown(void);
void logger_destroy(void);

#endif
```

logger.c (trechos principais)

```
/* Semaforos POSIX */
static sem_t sem_espacos_log; /* espacos livres */
static sem_t sem_itens_log;   /* itens pendentes */
static pthread_mutex_t mutex_log = PTHREAD_MUTEX_INITIALIZER;

void logger_init(void) {
  clock_gettime(CLOCK_MONOTONIC, &tempo_inicio);
  sem_init(&sem_espacos_log, 0, TAMANHO_FILA_LOG);
  sem_init(&sem_itens_log,  0, 0);
```

```
}

void log_evento(TipoEvento tipo, int id, int val, int qtd) {
  if (sem_trywait(&sem_espacos_log) != 0) return; /* descarta */
  EventoLog ev = {tipo, id, val, qtd, ms_desde_inicio()};
  pthread_mutex_lock(&mutex_log);
  fila_log[cauda_log] = ev;
  cauda_log = (cauda_log + 1) % TAMANHO_FILA_LOG;
  pthread_mutex_unlock(&mutex_log);
  sem_post(&sem_itens_log);
}

static void *thread_log_func(void *arg) {
  while (1) {
    sem_wait(&sem_itens_log);
    pthread_mutex_lock(&mutex_log);
    EventoLog ev = fila_log[cabeca_log];
    cabeca_log = (cabeca_log + 1) % TAMANHO_FILA_LOG;
    pthread_mutex_unlock(&mutex_log);
    sem_post(&sem_espacos_log);
    if (ev.tipo == LOG_SHUTDOWN) break;
    /* exibe o evento com timestamp */
  }
  return NULL;
}
```

main.c (estrutura de terminação)

```
int main(void) {
  buffer_init(); logger_init(); logger_iniciar_thread();
  /* Cria consumidoras primeiro, depois produtoras */
  for (i = 0; i < NUM_CONSUMIDORES; i++)
    pthread_create(&tc[i], NULL, consumidor, &idc[i]);
  for (i = 0; i < NUM_PRODUTORES; i++)
```

```
    pthread_create(&tp[i], NULL, produtor, &idp[i]);
  /* Join produtores primeiro */
  for (i = 0; i < NUM_PRODUTORES; i++)
    pthread_join(tp[i], NULL);
  /* Join consumidores (garante consumo completo do buffer) */
  for (i = 0; i < NUM_CONSUMIDORES; i++)
    pthread_join(tc[i], NULL);
  /* Encerra o logger (flush da fila apos todo processamento) */
  logger_shutdown();
  buffer_destroy(); logger_destroy();
  return 0;
}
```